# Module: AI Integration Lite — APIs, ML Demo & Simple UI Patterns

**Status: Optional enrichment.** Do this only after you have: (1) a deployed or Lab-07-verified Django project, (2) Required MySQL labs comfortable for CRUD+JOIN, and (3) a GitHub portfolio with standard deliverables.
**Previous module:** Testing, Debugging & Clean Code *(or Placement if AI is deferred)*
**Next module:** Career, Resume & Interview Prep

> **Placement rule:** A missing AI demo never blocks job-ready status. A missing deployed Django app or weak SQL does.

**Module Outcome:**

- Optional: call one AI/ML API or demo from a form you already shipped
- Never let this module delay a deployed Django app or SQL comfort
- Keep secrets in env — the API key is not a GitHub souvenir

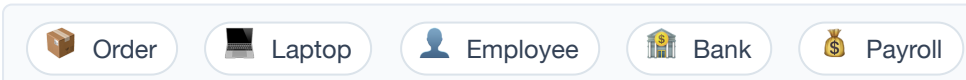
## Prerequisites

- **Optional stretch module** — only after Flask/Django API + a simple frontend form/ `fetch`
- `pip install scikit-learn` (or course Flask/Django project ready)

### AI in full-stack jobs (India 2026 realistic view)

Most **junior Python full-stack** roles want you to:

- Connect an app to an **existing** AI/ML service (OpenAI, Azure, company internal API)
- Send user text or data, display AI response in UI
- Handle errors, loading states, API keys safely




**Where used?**
- **TechMart** — canteen review sentiment / spam filter behind a Flask route
- College apps — simple chatbot FAQ endpoint on the student portal
- CRM / support — summarize tickets via remote model API
- **SecureBank** — fraud/risk scores consumed as JSON (you wire the call, not train GPT)
- Resume — one honest “AI-integrated demo” after Django deploy is solid

Employers do **not** expect you to publish research papers or build GPT from scratch. This module matches **brochure "AI Integration Lite"** — practical wiring.

### Two patterns we teach

| Pattern | Example | When |
|---|---|---|
| **Local tiny ML** | scikit-learn spam classifier | Offline demo, interviews "I know fit/predict" |
| **Remote model API** | HTTP POST to hosted LLM | Chatbot, summarizer in product |

Both end the same way for your React/Flask app: **JSON in, JSON out**.

### Architecture diagram

```
[User browser]
     |  text
     v
[Flask/Django endpoint]  --->  [Model: local sklearn OR remote API]
     |
     v
  JSON { "label": "positive", "score": 0.92 }
```

## Concepts

### scikit-learn minimal flow

1. **Train** on small dataset (`fit`)
2. **Predict** new sample (`predict` or `predict_proba`)
3. **Serve** result via web endpoint

**Note:**
- Expecting ML magic on 6 samples → wrong predictions — explain toy-model limits.
- Training on every request → slow server — train once, load the model at startup.

Build a tiny MultinomialNB text classifier on four campus reviews and predict a label for "amazing thali".

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

texts = ["great food", "cold coffee", "loved the dosa", "bad service"]
labels = [1, 0, 1, 0]  # 1=positive

vec = CountVectorizer()
X = vec.fit_transform(texts)
model = MultinomialNB().fit(X, labels)
print(model.predict(vec.transform(["amazing thali"])))
```

**Expected Output:** (label may be `1` = positive on this toy set)

```
[1]
```

**Remember:** Toy models on 4–6 samples are for demos — say so in interviews; production needs more data and evaluation.

**Common Error:** Hardcoding `API_KEY = "sk-..."` in source → leaked key on GitHub. Always `os.environ.get(...)`.

**Interview:** "Did you train GPT?" → No — "I called a hosted model API / tiny sklearn classifier from Flask/Django and returned JSON to the UI."

Write a short markdown note describing the train/predict flow and the rule to never ship API keys in code.

```
# Flow

1. Collect labeled reviews (campus canteen)
2. Vectorize text → numbers
3. Fit model
4. predict() on new review
Never ship API keys in code.
```

## Calling remote AI API (pattern)

```
import os
import requests

API_KEY = os.environ.get('MODEL_API_KEY')  # never hardcode

headers = {'Authorization': f'Bearer {API_KEY}'}
payload = {'input': user_text}
resp = requests.post('https://api.example.com/v1/analyze', json=payload, headers=headers, timeout=30)
resp.raise_for_status()
return resp.json()
```

Course may use mock URL in lab if no paid API key — pattern matters.

**Note:**
- Blocking call without timeout → hung request — use `timeout=30` on requests.

Load the AI API key from an environment variable and exit with a clear message if it is missing.

```
import os

API_KEY = os.environ.get("AI_API_KEY")
if not API_KEY:
    raise SystemExit("Set AI_API_KEY in environment — do not hardcode")
print("Key loaded:", bool(API_KEY))
```

Implement a mock `analyze` function that returns positive or neutral based on keywords — no network call required.

```
def mock_analyze(text: str) -> dict:
    lowered = text.lower()
    label = "positive" if "good" in lowered or "love" in lowered else "neutral"
    return {"text": text, "label": label, "source": "mock"}
```

```
print(mock_analyze("Good filter coffee in Hyderabad"))
```

## Text analyzer UI pattern (React or template)

1. Textarea + Submit button
2. `loading` state true during fetch
3. Display result or error message
4. Disable double-submit

**Note:**
- No input validation → crash or abuse — add length limits and strip text.
- CORS again on AI Flask → browser block — reuse the same cors-headers pattern.

Write an async function that POSTs review text to `/api/analyze` and returns parsed JSON from the response.

```
async function analyzeReview(text) {
  const res = await fetch("/api/analyze", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ text }),
  });
  if (!res.ok) throw new Error("analyze failed");
  return res.json();
}
```

Sketch a React component with loading state, result display, and protection against double-submit while fetching.

```
import { useState } from "react";

export default function Analyzer() {
  const [loading, setLoading] = useState(false);
  const [result, setResult] = useState(null);
  async function onSubmit(text) {
    setLoading(true);
    try {
      setResult(await analyzeReview(text));
    } finally {
      setLoading(false);
    }
  }
  return (
    <div>
      {loading ? <p>Analyzing…</p> : null}
      {result ? <pre>{JSON.stringify(result, null, 2)}</pre> : null}
    </div>
  );
}
```

## Security basics

- `.env` in `.gitignore`
- `.env.example` with placeholder keys
- Rate limit mention (production)
- Do not log full user PII

**Note:**
- API key in GitHub → key revoked / security incident — use `.env` + gitignore.
- Overclaiming on resume → interview embarrassment — say "integrated API," not "built GPT."

Add `.env`, pem files, and service-account JSON to `.gitignore` so secrets are never committed.

```
.env
*.pem
service-account.json
```

Draft a short ethics note for your portfolio README covering PII, mock vs real models, and rate limiting.

```
# AI ethics (portfolio README)

- Do not paste customer PII into public models
- Label mock vs real model clearly
- Rate-limit `/api/analyze` in production
```

## Step-by-Step Annotated Code

### Part A — scikit-learn sentiment-style demo (tabular)

`ml/train_sentiment_demo.py`:

```
"""Train tiny model on sample data — educational only."""
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

# Toy data: 0 = negative, 1 = positive
texts = [
    'I love this product', 'Amazing service', 'Great experience',
    'Terrible quality', 'Worst support ever', 'Very bad experience',
]
labels = [1, 1, 1, 0, 0, 0]

model = Pipeline([
    ('vect', CountVectorizer()),
    ('clf', LogisticRegression(max_iter=1000)),
])
model.fit(texts, labels)

# Quick test
print(model.predict(['good job']))        # expect [1]
print(model.predict(['awful delay']))     # expect [0]

# Optional: save for Flask
import joblib
joblib.dump(model, 'ml/sentiment_model.joblib')
```

### Part B — Flask prediction endpoint

`ai_app/app.py`:

```
import os
from flask import Flask, request, jsonify
import joblib

app = Flask(__name__)
model = joblib.load('ml/sentiment_model.joblib')

@app.route('/api/health')
def health():
    return jsonify({'status': 'ok', 'service': 'sentiment-api'})

@app.route('/api/analyze', methods=['POST'])
def analyze():
    data = request.get_json()
    if not data or 'text' not in data:
        return jsonify({'error': 'text field required'}), 400
    text = data['text'].strip()
    if not text:
        return jsonify({'error': 'text cannot be empty'}), 400
    if len(text) > 5000:
        return jsonify({'error': 'text too long'}), 400

    prediction = int(model.predict([text])[0])
    label = 'positive' if prediction == 1 else 'negative'
    return jsonify({'label': label, 'text_length': len(text)})
```

Test:

```
curl -X POST http://127.0.0.1:5000/api/analyze \
  -H "Content-Type: application/json" \
  -d '{"text":"I love this course"}'
```

### Part C — Django equivalent (sketch)

Use `@api_view(['POST'])`, load joblib model once, return `Response({'label': ...})` — same pattern as Flask.

### Part D — React analyzer UI (pattern)

Form with textarea → `fetch POST /api/analyze` → show loading → display `label` or error. Disable submit while loading; validate empty text client-side.

Flask route returns `{'reply': mock_chat_reply(text)}` — same UI swap as analyze; replace with real LLM API in production.

---

## Practice Exercises

### Basic

1. Hit `/api/health` and document response.
   **Model answer:**

```
curl -s http://127.0.0.1:5000/api/health
```

Expect JSON like `{"status": "ok"}` (or similar). Paste status code + body into your lab notes.

2. Send 3 analyze requests — one positive, one negative, one empty (400).
   **Model answer:**

```
curl -s -X POST http://127.0.0.1:5000/api/analyze -H 'Content-Type: application/json' -d '{"text":"grea
curl -s -X POST http://127.0.0.1:5000/api/analyze -H 'Content-Type: application/json' -d '{"text":"terr
curl -s -X POST http://127.0.0.1:5000/api/analyze -H 'Content-Type: application/json' -d '{"text":""}'
```

Empty text should return **400** with an error message.

3. Add `.env.example` with `MODEL_API_KEY=your_key_here`.
   **Model answer:**

```
echo 'MODEL_API_KEY=your_key_here' > .env.example
```

Commit `.env.example` only — never the real `.env`.

## Intermediate

1. Add `score` using `predict_proba` in Flask response.
   **Model answer:**

```
proba = model.predict_proba([text])[0]
label = model.classes_[proba.argmax()]
return jsonify({"label": str(label), "score": float(proba.max())})
```

2. Chat UI with message history displayed as list.
   **Model answer:**
   Keep a `messages` array in React/JS state (or server session). On send, append user text, call the API, append assistant reply, and render `<ul>{messages.map(...)}</ul>` (or equivalent).
3. Log analyze requests at INFO without logging full text (privacy).
   **Model answer:**

```
logger.info("analyze request len=%s", len(text))
# do NOT log: logger.info(text)
```

## Interview-style

1. Difference between training model and calling model API?
   **Model answer:**
   Training fits model weights on a dataset (compute-heavy, offline). Calling a model API sends inputs to an already-trained model and gets predictions — inference only, usually via HTTP.
2. Where store OpenAI key in Django project?
   **Model answer:**
   In environment variables (or a secrets manager), loaded into settings — never hardcode in source or commit `.env`. Example: `OPENAI_API_KEY = os.environ["OPENAI_API_KEY"]`.
3. One risk of sending user data to third-party AI?
   **Model answer:**
   Privacy/compliance risk: personal or confidential text may be stored or used by the vendor. Minimize data sent, get consent where required, and avoid PII in prompts when possible.

## Mini-Project (Assignment): Review Analyzer Widget

**Lab kit (build here):** 15_Lab_Assets/29_ai_review_stub/

**Goal:** End-to-end lite AI feature for portfolio.

### What to build

- Trained sklearn model OR documented mock
- Flask or Django POST `/api/analyze`
- Simple UI (React or HTML) with loading state
- README: architecture diagram, ethics note, how to run
- No secrets in repo

### Stretch

- Swap mock chatbot to real free-tier API only if your batch allows external API keys.
- Add sample Indian-English reviews (dosa, thali, filter coffee).

### Done when

- [ ] Lab kit optional AI recommend ticket completed (or mock path documented)
- [ ] `/api/analyze` returns JSON label
- [ ] UI shows loading + result
- [ ] No API keys committed; ethics note in README

## Interview Q&A

Shared with students after concepts and the mini-project. Use for review and interview prep — questions and model answers are included.

1. Should every app add AI because it is trending?
2. What user consent might you need before sending reviews to cloud AI?
3. how is calling ML API like using payment gateway SDK?
4. Why is toy sentiment model still useful in training if not production accurate?
5. How would you explain your mini-project in 30 seconds to interviewer?

**Q1. Have you worked with AI/ML in projects?**
**A:** Integrated sentiment analysis via Flask API using scikit-learn model and React UI; understand calling external model APIs with env-based keys.

**Q2. scikit-learn used for?**
**A:** Traditional ML in Python — classification, regression, clustering; good for tabular/text baselines.

**Q3. How protect API keys?**
**A:** Environment variables, secrets manager in cloud, never commit to Git, rotate if leaked.

**Q4. REST vs AI-specific — same?**
**A:** Same HTTP JSON pattern; AI endpoint returns inference result instead of CRUD resource.

**Q5. Risks of AI features?**
**A:** Wrong answers, bias, privacy, cost, dependency on third party; need disclaimers and validation.